

```

import os
import json
import sqlite3
import hashlib
import psycopg2
from psycopg2.extras import RealDictCursor
import time
import logging

# Configuration
PG_CONN = os.getenv('PG_CONN',
"postgres://postgres:password@localhost:5432/keystone")
SQLITE_PATH = os.getenv('SQLITE_PATH', "./data/history.db")
REPLICATION_SLOT = os.getenv('REPLICATION_SLOT', "keystone_slot")

# Setup Logging
logging.basicConfig(level=logging.INFO, format='%(asctime)s -
%(levelname)s - %(message)s')
logger = logging.getLogger(__name__)

class SQLiteStore:
    def __init__(self, path):
        self.conn = sqlite3.connect(path)
        self.conn.row_factory = sqlite3.Row
        self.setup()

    def setup(self):
        with self.conn:
            self.conn.execute("PRAGMA journal_mode = WAL")
            self.conn.execute("PRAGMA synchronous = NORMAL")
            self.conn.execute("""
                CREATE TABLE IF NOT EXISTS events (
                    event_id TEXT PRIMARY KEY,
                    sha256 TEXT UNIQUE,
                    timestamp TEXT,
                    event_type TEXT,
                    payload_digest TEXT,
                    payload_preview TEXT,
                    wal_lsn TEXT
                )
            """)
            self.conn.execute("CREATE TABLE IF NOT EXISTS metadata (k
TEXT PRIMARY KEY, v TEXT)")
            self.conn.execute("INSERT OR IGNORE INTO metadata(k,v)
VALUES ('last_applied_lsn','0/0')")

    def get_last_lsn(self):
        cur = self.conn.cursor()

```

```

        cur.execute("SELECT v FROM metadata WHERE
k='last_applied_lsn'")
        row = cur.fetchone()
        return row['v'] if row else '0/0'

def apply_event(self, event):
    try:
        with self.conn:
            self.conn.execute("""
                INSERT INTO events (event_id, sha256, timestamp,
event_type, payload_digest, payload_preview, wal_lsn)
                VALUES (?, ?, ?, ?, ?, ?, ?)
            """, (
                event['event_id'], event['sha256'],
event['timestamp'],
                event['event_type'], event['sha256'],
event['preview'], event['wal_lsn']
            ))
            self.conn.execute("UPDATE metadata SET v = ? WHERE k =
'last_applied_lsn'", (event['wal_lsn'],))
            return True
    except sqlite3.IntegrityError:
        return False

def sanitize_and_hash(payload):
    # Strip PII
    sanitized = payload.copy()
    for key in ['user_ssn', 'user_email']:
        if key in sanitized:
            sanitized[key] = '[REDACTED]'

    encoded = json.dumps(sanitized, sort_keys=True).encode()
    sha256 = hashlib.sha256(encoded).hexdigest()
    preview = json.dumps(sanitized)[:1024]
    return sha256, preview

def run_bridge():
    store = SQLiteStore(SQLITE_PATH)

    # Initialize PG connection
    try:
        pg_conn = psycopg2.connect(PG_CONN)
        pg_conn.autocommit = True
        cur = pg_conn.cursor(cursor_factory=RealDictCursor)

        # Ensure slot
        cur.execute("SELECT count(*) FROM pg_replication_slots WHERE
slot_name=%s", (REPLICATION_SLOT,))

```

```

        if cur.fetchone()['count'] == 0:
            cur.execute("SELECT pg_create_logical_replication_slot(%s,
'wal2json')", (REPLICATION_SLOT,))
            logger.info(f"Created replication slot:
{REPLICATION_SLOT}")

        while True:
            last_lsn = store.get_last_lsn()
            # Poll changes
            cur.execute("SELECT data, lsn FROM
pg_logical_slot_get_changes(%s, NULL, NULL, 'pretty-print', '1')",
(REPLICATION_SLOT,))
            rows = cur.fetchall()

            if not rows:
                time.sleep(1)
                continue

            for row in rows:
                data = json.loads(row['data'])
                lsn = row['lsn']

                for change in data.get('change', []):
                    if change.get('table') == 'events' and
change.get('kind') == 'insert':
                        # Map columns
                        cols = change.get('columnnames', [])
                        vals = change.get('columnvalues', [])
                        record = dict(zip(cols, vals))

                        sha, preview =
sanitize_and_hash(record.get('payload', {}))

                        event_data = {
                            'event_id': record['event_id'],
                            'sha256': sha,
                            'timestamp': record['timestamp'],
                            'event_type': record['event_type'],
                            'preview': preview,
                            'wal_lsn': lsn
                        }

                        if store.apply_event(event_data):
                            logger.info(f"Applied event
{event_data['event_id']} at {lsn}")

            except Exception as e:
                logger.error(f"Bridge failure: {e}")

```

```
        time.sleep(5)

if __name__ == "__main__":
    run_bridge()
```